Data visualisation in RStudio using ggplot2

Anders Jensen

24/03/26

University of Liverpool

Introduction

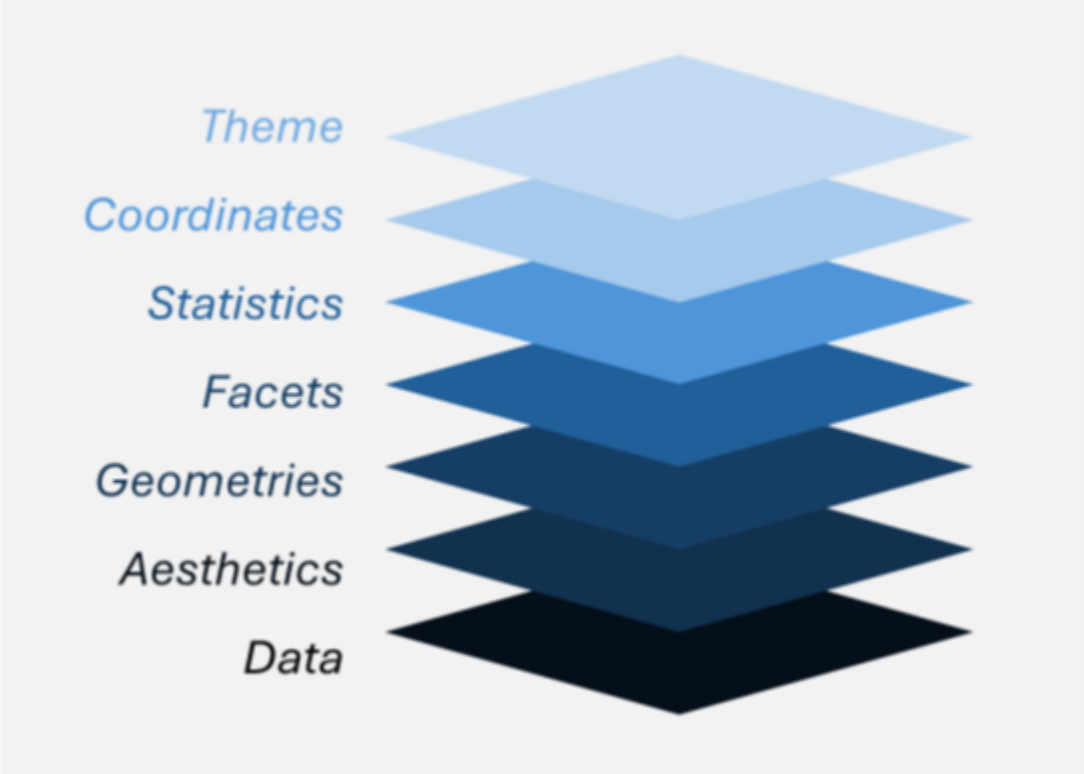
One of the most important stages on all data analysis pipelines is understanding how to best visualise your data so that they can be easily understood and represented. The challenge of presenting our research and findings is made much simpler with aesthetically pleasing diagrams and figures that keep audiences informed and engaged.

The biggest problems we may face with data visualisation include, determining which type of graph to use and how to then produce the figure. Understanding which type of graph to use comes with experience and considering two main factors; What is the important part of your data? What type of variables are you trying to show? In this workbook we will go through some examples of how to decide on the type of graph. This workbook will also provide the framework for how to make these plots using Rstudio and specifically a package called GGPlot2.

GGPlot2 is a highly popular R package invented by **Hadley Wickham** in 2005 and is used to design graphics for people utilising R programming language. There is a built in system for making plots in R, however GGPlot2 is much more effective, due to simplicity and ability to control more plot features.

GGPlot2 aims to implement **Leland Wilkinsons Grammer** of Graphics which is a book that was released in 2005 and is a fascinating insight into how all graphics can be split into certain components. The following youtube video explains these principles well for those who may be interested. For anyone really interested here is the link to original book.

Before attempting this guide it is recommended that you work through the introduction to R handbook first.



Installing and loading ggplot2

This should be very simple if you have worked through the previous handbook but as a reminder:

To install a package in Rstudio you need to run install.packages("name_of_package") and then run library(name_of_package), see below.

```
install.packages("ggplot2")
library(ggplot2)
```

Remember you only need to install the package once, however you need to load it in everytime you start a new R session.

The fundamentals of a ggplot

The data

To begin producing a graphic you first need a dataset to work with. For the purposes of this workbook we will be using datasets that are already built into R. To look at the available datasets you can run the following code:

```
data()
```

This should show up a list of datasets that you can use, for example: AirPassengers or ChickWeight

To then save one of these datasets into your environment you need to run the following code

```
data('iris')
```

Have a go at loading in the iris dataset into your environment. This dataset is Edgar Andersons famous data on the variable's sepal length and width and petal length and width, respectively, for 50 flowers from each of 3 species of iris. The species are Iris setosa, versicolor, and virginica.

The type of graph

There are too many types of graphs to list, however some of the most common include scatter plot, box plot, line plot, bar plot, histogram etc.

Understanding when to use each of these comes from thinking about the information you are trying to portray as well as the nature of the variables you are reporting.

Are your variables continuous, categorical, ordinal or logical?

The following table contains information about when these plots may be appropriate and the type of data that may suit them.

Common Data Visualisation Plots

Plot Type	Variables Required	What It Shows
Scatter plot	Two numeric continuous variables	Shows two values on an x and y axis
Box plot	One numeric variable and one continuous variable	This allows you to show a continuous variable amongst different groups. You can also visualise distribution
Line plot	One numeric variable on the Y-axis then either a ordinal or continuous variable	Useful for tracking a value over another constant (time, concentration etc)
Bar plot	One numeric variable and one continuous variable	Similar to boxplot but shows less information. Can be easier to interpret
Histogram	One continuous or ordinal variable	Shows the distribution of data

ggplot2 basic code

This is the most important part. How to begin writing ggplot2 code.

You can think of ggplot2 as a cake where you can keep on adding ingredients to make it more complex. No matter how good the cherry on top looks, you still need flour, eggs and milk.

The following line of code is the backbone of every ggplot and should always be your starting point whenever making a plot. You will see this a lot more moving forward in this document so take the time to familiarise yourself.

```
ggplot(data, aes(x = x_axis, y = y_axis))
```

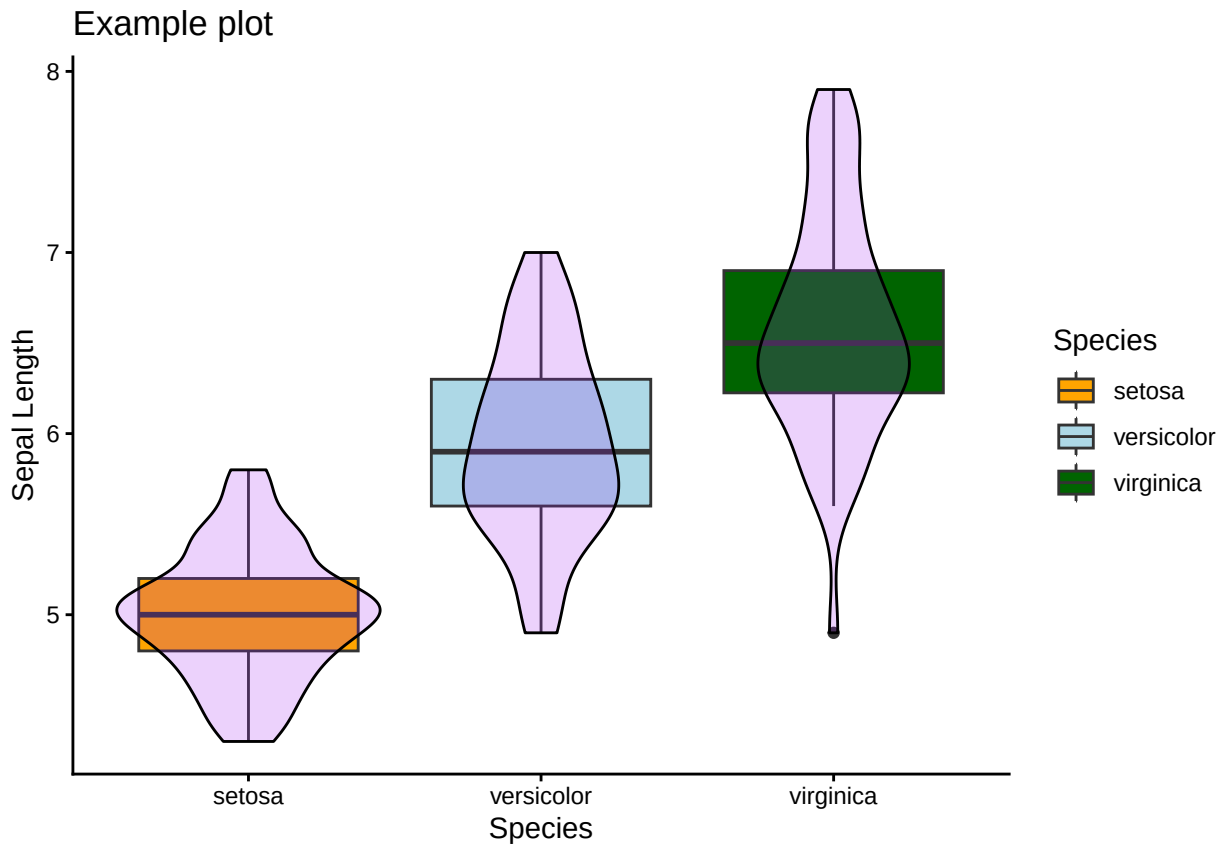
So lets break this code down:

- **ggplot** – This is the ggplot2 function to say you are wanting to make a graphic
- **data** – this is the name of the data frame saved in your environment
- **aes** – this is a function to state that you are looking inside the data frame
- **x** – this is the column name in your data frame you want on the x-axis of the plot
- **y** - this is the column name in your data frame you want on the x-axis of the plot

Then if you wanted to add extra information to the ggplot2 you would use a + sign and write a new line of code. This is where you can add information about the type of plot.

The plot below is an example of how this works, hopefully by the end of this workbook you can make plots that look so much better than this.

```
ggplot(iris, aes(x = Species, y = Sepal.Length))+
  geom_boxplot(aes(fill = Species))+
  geom_violin(alpha = 0.2, fill = "purple", col = "black")+
  theme_classic()+
  scale_fill_manual(values = c("orange", "lightblue", "darkgreen"))+
  labs(x = "Species", y = "Sepal Length", title = "Example plot")
```



Making a scatter plot

We will begin with making a simple scatter plot. A scatter plot is appropriate for when you are trying to graph two continuous variables against each other, such as height and weight.

For this example, we will use the 'iris' dataset. For more information on this data, you can run the following code.

```
?iris
```

Firstly, we will load in the data

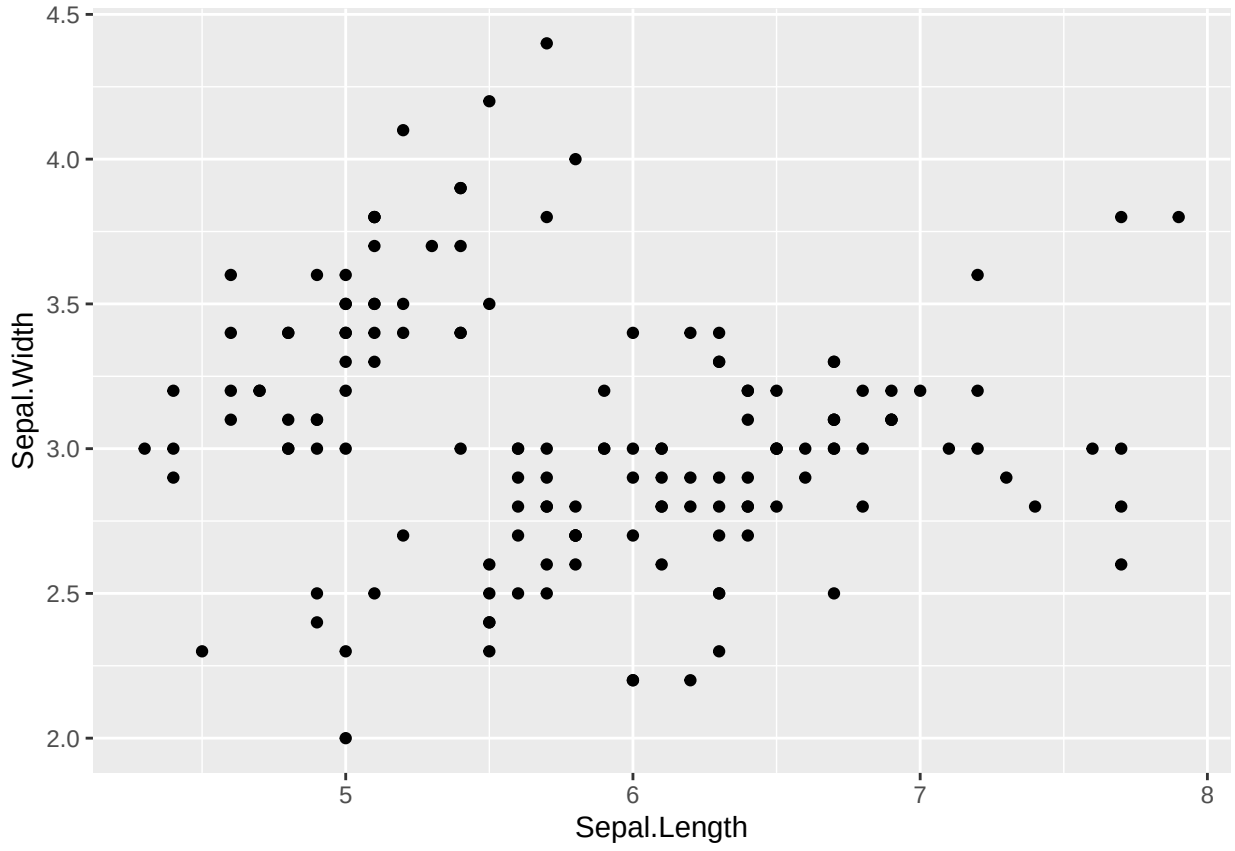
```
# Load data
data('iris')
```

Then we will use the code from above to build a blank ggplot

```
# make blank ggplot
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))
```

Then we will use the + sign to add some points to the graph

```
# add in the points
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))+
  geom_point()
```

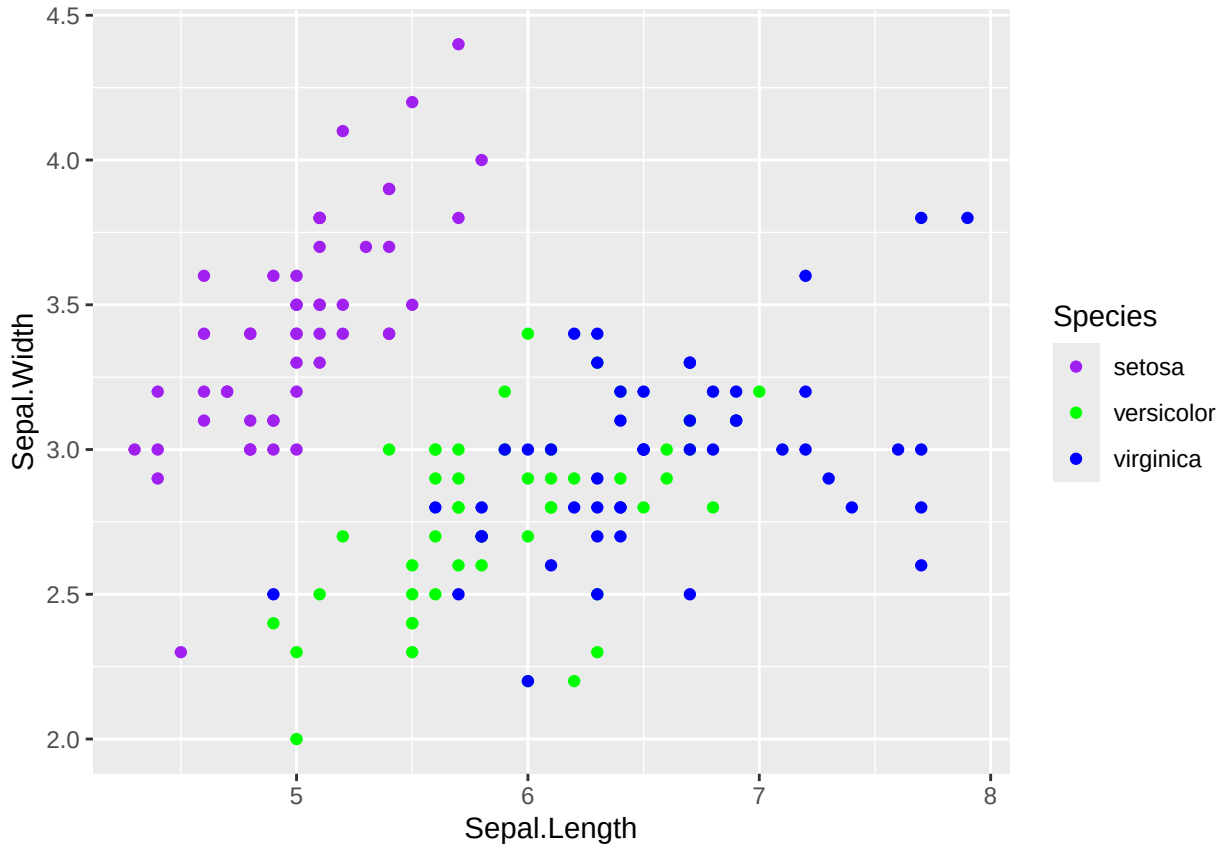


Hopefully if you followed this code, you should see a simple ggplot in the plots window with Sepal width on the y-axis and Sepal length on the x-axis.

Make sure column names are spelt correctly, and brackets are closed correctly.

We will now change the colour of the points depending on the species.

```
# change colour of points
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))+
  geom_point(aes(col = Species)) + # This tells R to change colour based on species
  scale_color_manual(values = c("purple", "green", "blue")) # This lets you choose colours
```



Notice how we used col = Species inside of the aesthetics. This is because Species is within the dataset.

If we just wanted all the dots to be red then we wouldn't need aes() and the code would be:

```
# add points to plot
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))+
  geom_point(col = "red")
```

Finally, we can try changing both the size and the shape of the points.

```
# make blank ggplot
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))+
  geom_point(aes(col = Species),
             size = 10, # size of points
             shape = 21, # shape
```

```
            fill = "grey") + # the inner colour of points
  scale_color_manual(values = c("purple", "green", "blue"))
```

Again this plot is not the most visually appealing.

The different shape options can be seen below.

Figure 1: ggplot point shapes

Making a bar plot

A bar plot or bar chart is another kind of plot that is suitable for when you have a continuous variable and a categorical variable.

For this example, we will use the "InsectSprays"dataset. For more information on this data, you can run the following code

```
?InsectSprays
```

Firstly, we will load in the data

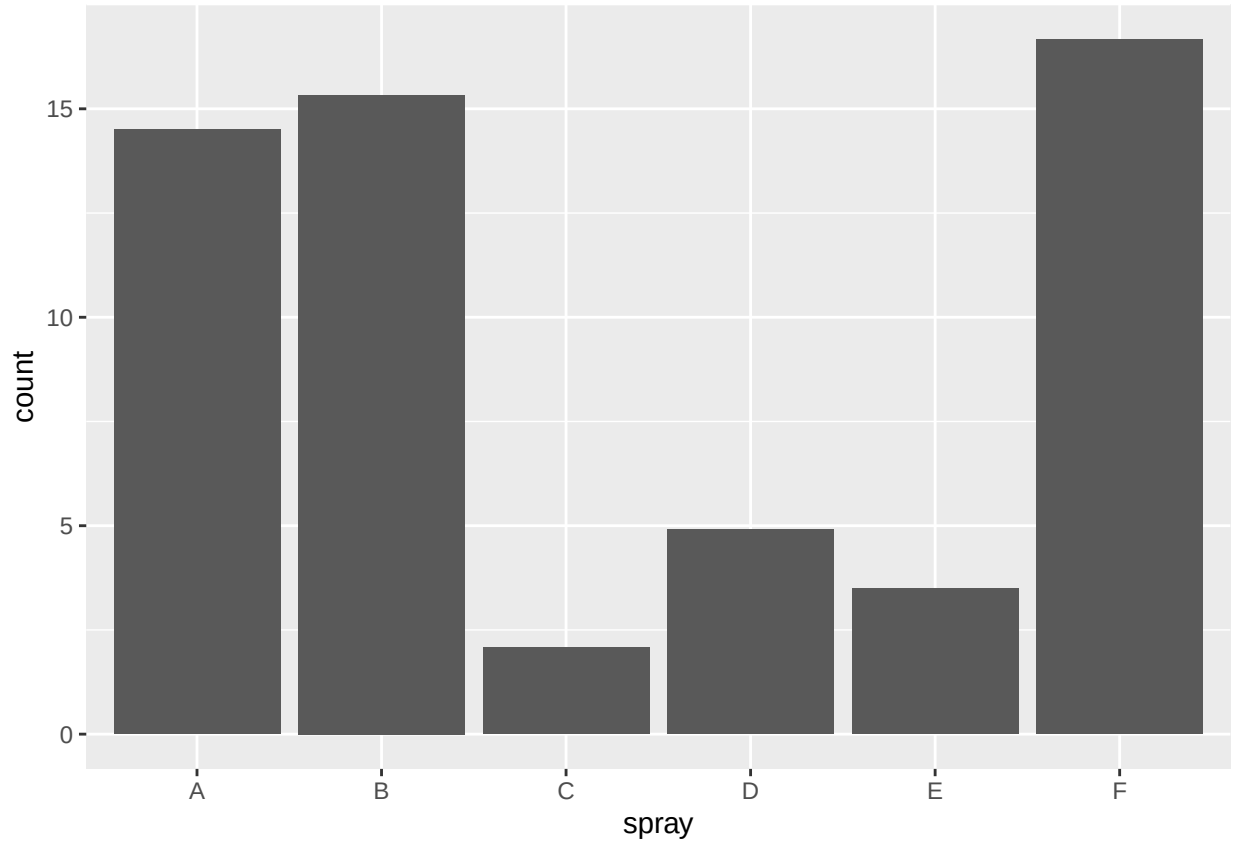
```
# Load data
data('InsectSprays')
```

Then we will use the code from above to build a blank ggplot

```
# make blank ggplot
ggplot(InsectSprays, aes(x = spray, y = count))
```

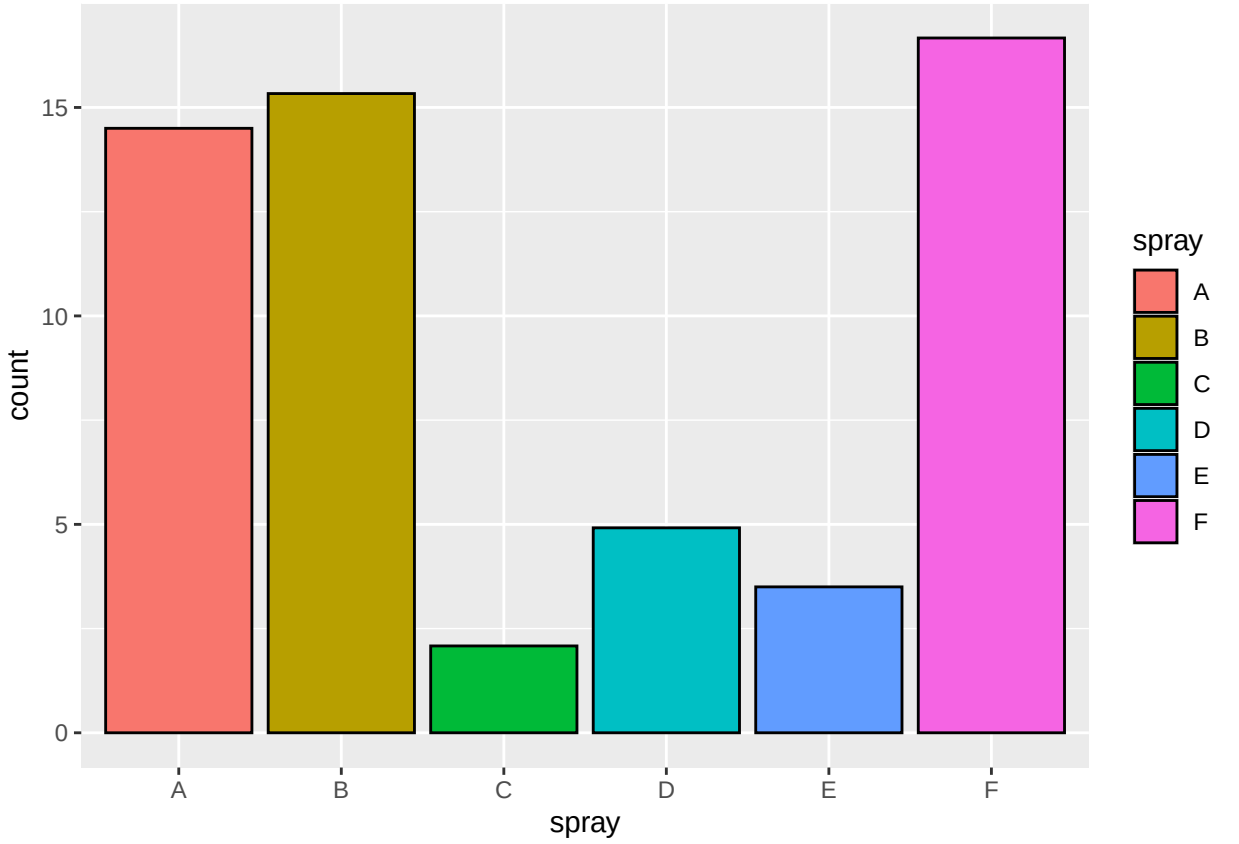
Then we will add the columns

```
# add bar to plot
ggplot(InsectSprays, aes(x = spray, y = count))+
  geom_bar(stat = "summary", fun = mean)
```



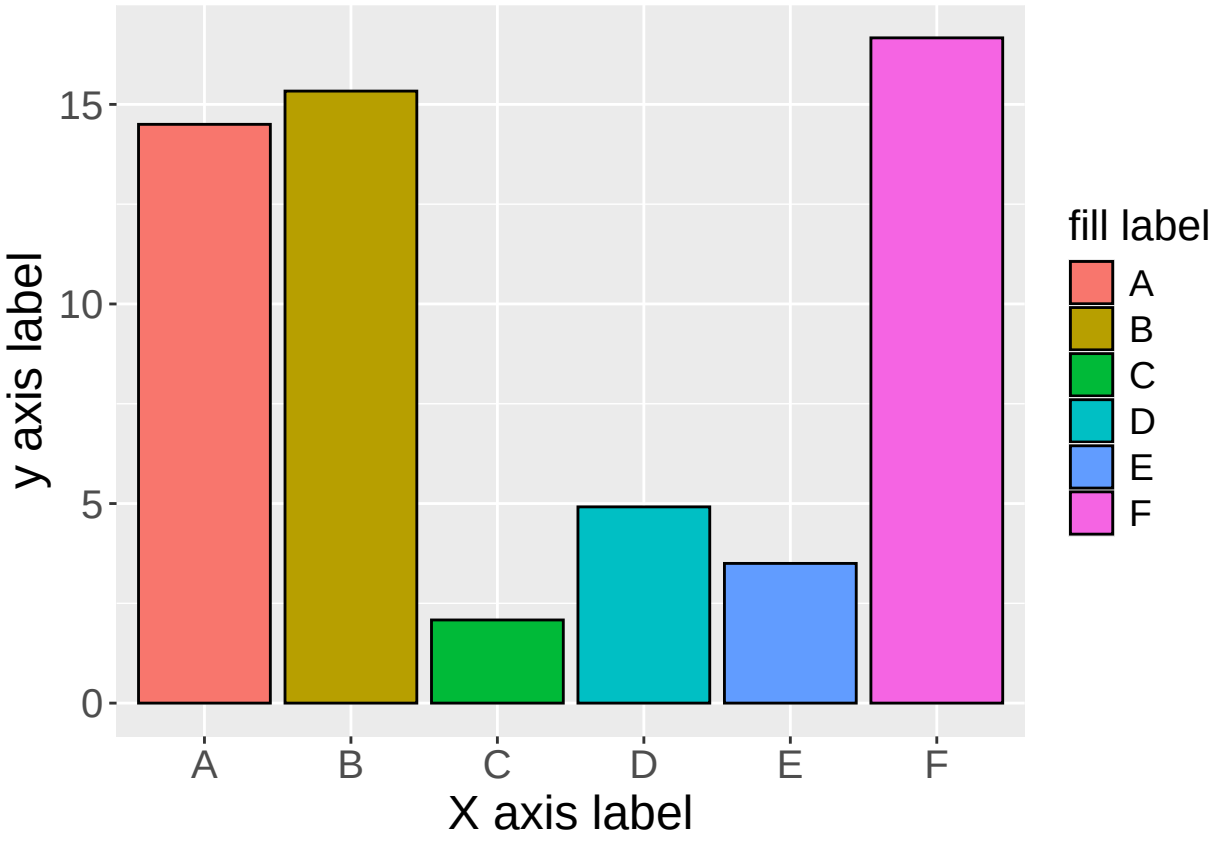
Then we can change colours

```
# change bar colours
ggplot(InsectSprays, aes(x = spray, y = count))+
  geom_bar(stat = "summary",
           fun = mean,
           col = "black", # outer colour of bars
           aes(fill = spray)) # inner colour of bars
```



Then we can change labels and size of text

```
# change bar colours
ggplot(InsectSprays, aes(x = spray, y = count))+
  geom_bar(stat = "summary",
           fun = mean,
           col = "black",
           aes(fill = spray)) +
  labs(x = "X axis label",
       y = "y axis label",
       fill = "fill label")+
  theme(axis.text = element_text(size = 15),
        axis.title = element_text(size = 18),
        legend.text = element_text(size = 14),
        legend.title = element_text(size = 16))
```



The labs() functions allows you to specify different labels on the graph and the theme() function allows you to control various aspects of your graphs.

The stat = summary, fun = mean arguments allow you to take long data and only plot the mean value, if you want to plot the sum you can change the code to fun = sum.

The theme() function allows you to change certain aspects of the plot. In this example we have changed the size of the text of the axis titles as well as the text on the axis themselves. You can also change the colour and font this way.

Making a histogram

A histogram is another kind of plot that is suitable for presenting the distribution of data. For this example, we will use the "ToothGrowth" dataset. For more information on this data, you can run the following code.

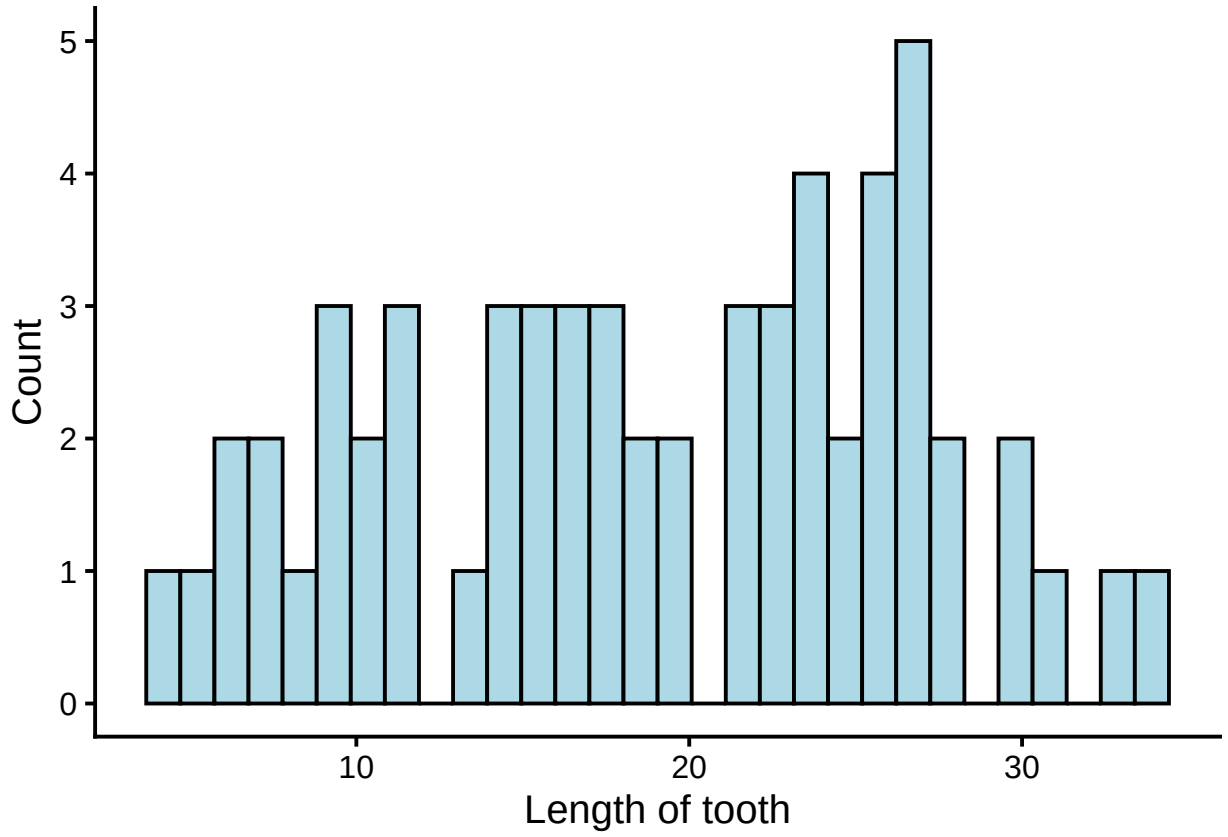
```
?ToothGrowth
```

The sequence of building this ggplot is the same as the ones before it.

1) Load data
2) Build blank ggplot
3) Specify the type of ggplot
4) Tidy up the plot

```
# load data
data("ToothGrowth")

# build plot
ggplot(ToothGrowth, aes(x = len))+
  geom_histogram(col = "black", fill = "lightblue")+
  theme_classic(base_size = 15)+
  labs(x = "Length of tooth", y = "Count")
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

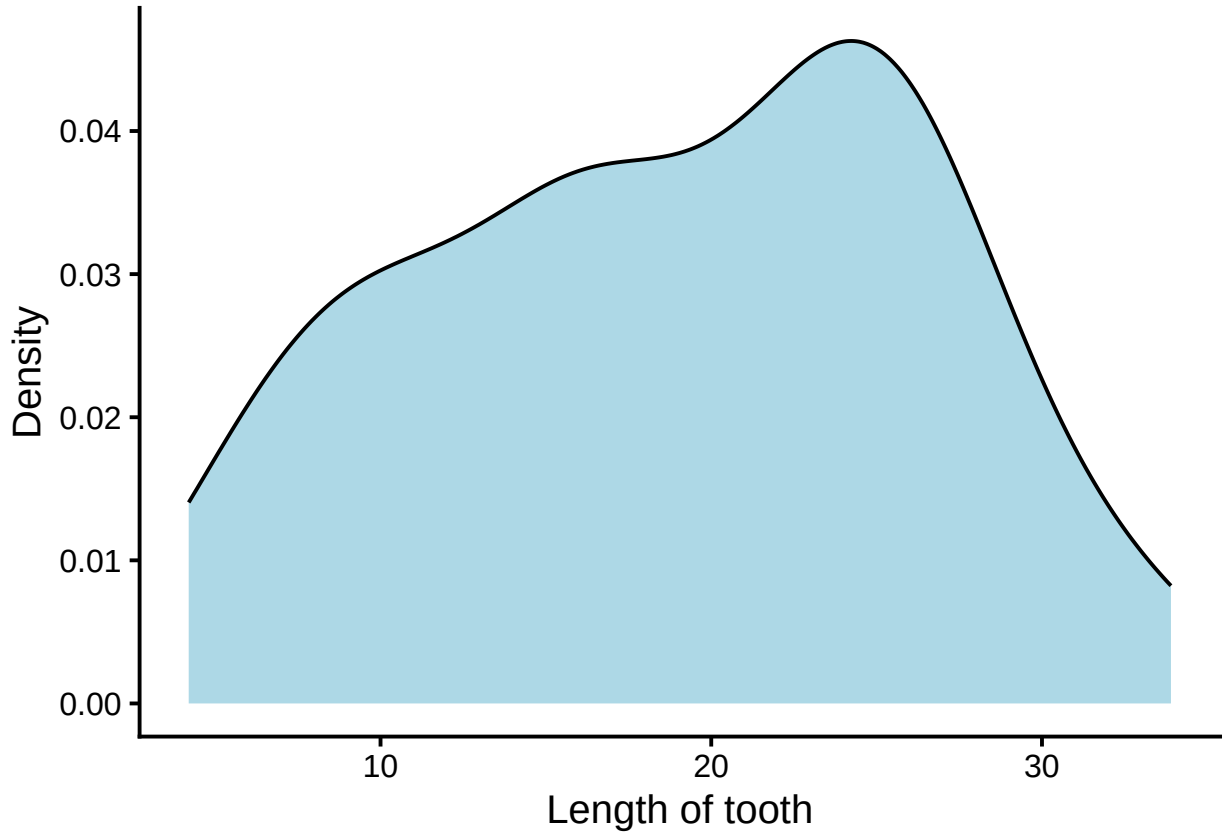


When plotting a histogram you don't need to specify the y-axis as this is assumed as the count when used the function geom_histogram.

While histograms are good at visualising data distribution, the Shapiro-wilk's test or the Kolmogorov-Smirnoff test may be preferred choices to quantitively test the normality of data.

Instead of a histogram you can also represent data as a density plot using the code below.

```
# load data
data("ToothGrowth")

# build plot
ggplot(ToothGrowth, aes(x = len))+
  geom_density(col = "black", fill = "lightblue")+
  theme_classic(base_size = 15)+
  labs(x = "Length of tooth", y = "Density")
```



Making a boxplot

A boxplot is another plot, which is similar to a barplot and can be used when you have one continuous variable and a categorical variable.

For this example, we will use the "starwars" dataset. For more information on this data, you can run the following code

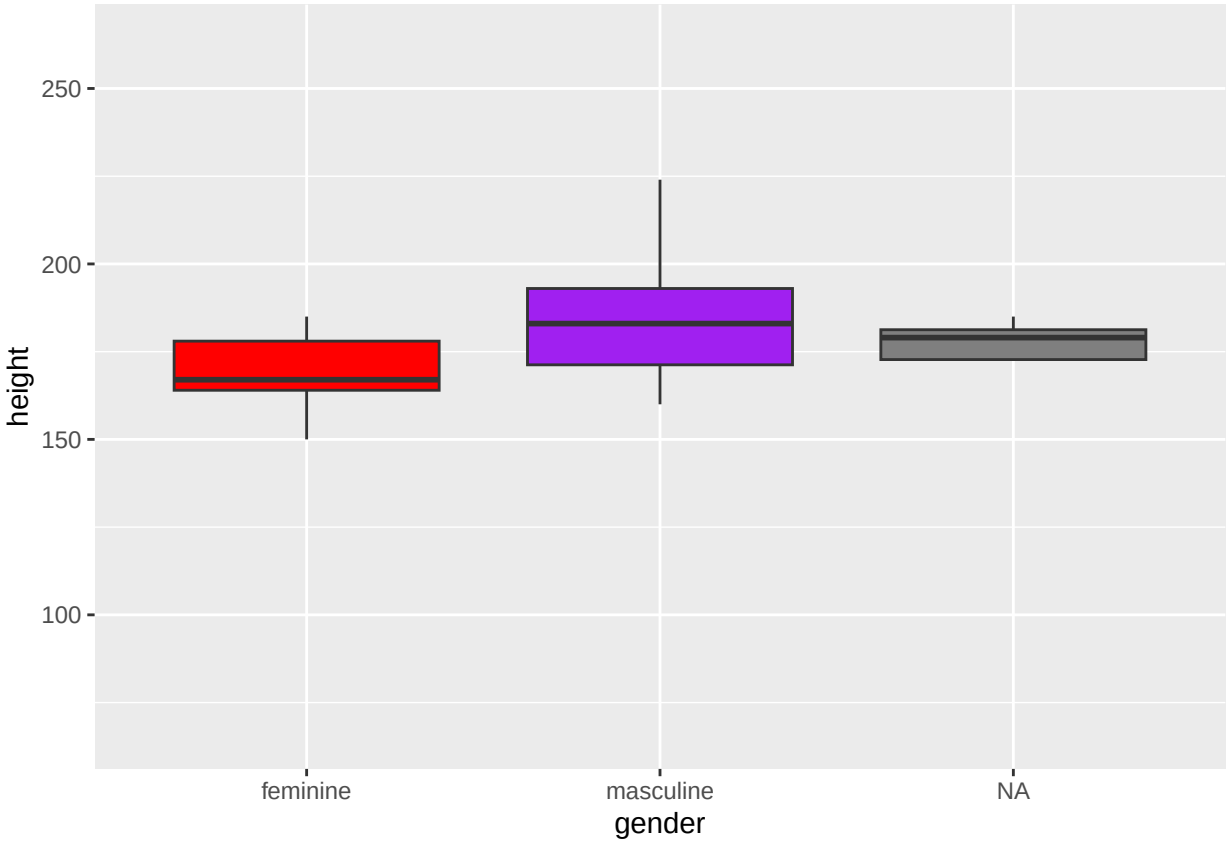
```
?starwars
```

The sequence of building this ggplot is the same as the ones before it.

1) Load data
2) Build blank ggplot
3) Specify the type of ggplot
4) Tidy up the plot

```
# load data
data("starwars")

# build plot
ggplot(starwars, aes(x = gender, y = height))+
  geom_boxplot(aes(fill = gender),
               show.legend = F,
               outlier.shape = NA)+
  scale_fill_manual(values = c("red", "purple"))
```

```
## Warning: Removed 6 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



The scale_fill_manual() function allows you to specify the colours for each of the boxes.

As a challenge have a go at changing the labels and the size of the labels in the plot. Hint: labs(), theme()

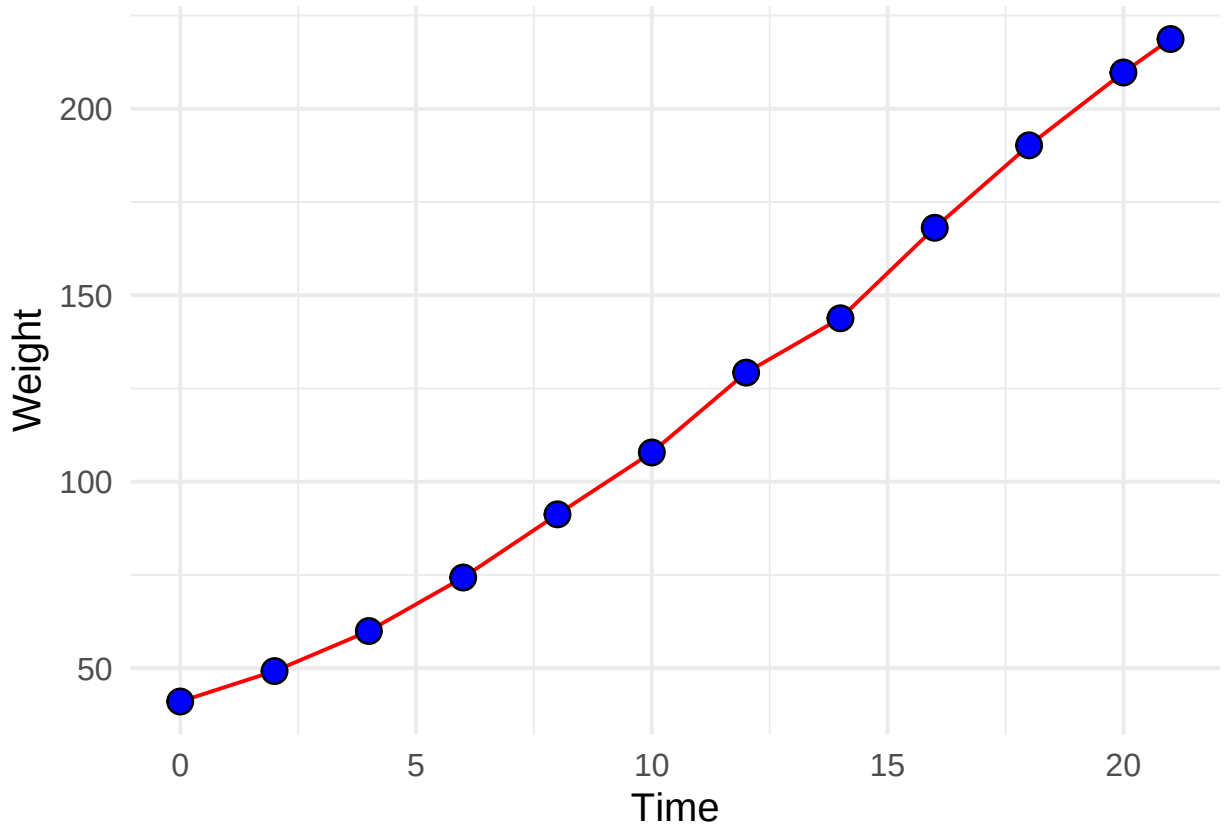
Making a line plot

A line plot is useful for representing a change in a continuous variable over a certain time, concentration or amount.

For this example, we will use the "ChickWeight" dataset. For more information on this data, you can run the following code.

```
?ChickWeight
```

```
# load in data
data("ChickWeight")

# transform data
ChickWeight_summarised <- ChickWeight %>%
  group_by(Time) %>%
  summarise(avg_weight = mean(weight, na.rm = T))

# build plot
ggplot(ChickWeight_summarised, aes(x = Time, y = avg_weight))+
  geom_line(col = "red")+
  geom_point(fill = "blue",
             col = "black",
             shape = 21,
             size = 4)+
  theme_minimal(base_size = 15)+
  labs(x = "Time", y = "Weight")
```



This is the first example of a plot where we had to do some data cleaning before building the plot. This was completed using the **dplyr** package which you will need to install and load first.

There is some basic information about dplyr near the end of this book. For further information the following youtube channel has many useful guides.

Additional ggplot features

Once you have built the basic ggplot there will be lots of subtle changes you may want to make to increase the readability and design of your plot.

Nearly every change you want to make will be achievable but may get slightly more complicated. Some possible problems are listed below but the best sources to find answers to problems are YouTube, stack overflow different AI tools.

The following code is used to make an example plot which will be used to answer the questions below.

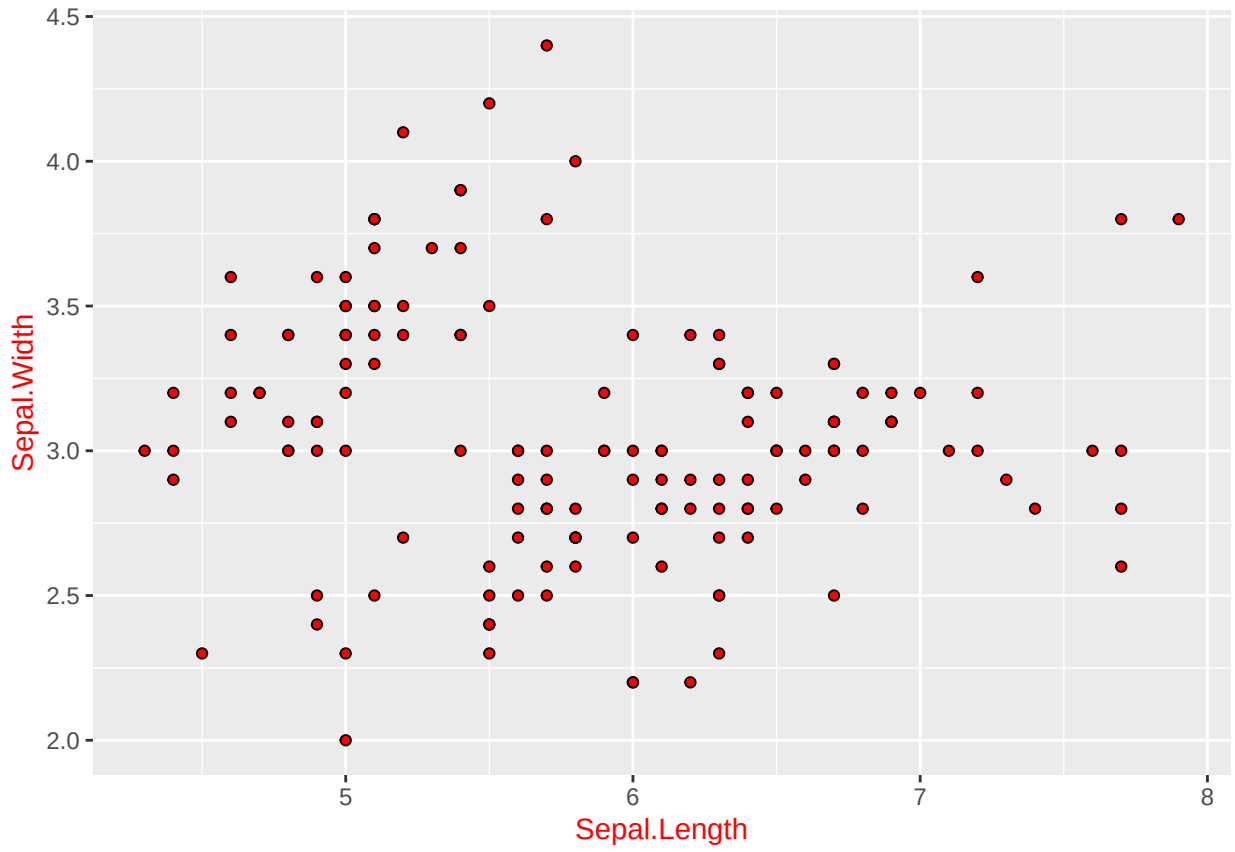
```
plot <- ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))

plot2 <- ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width))
```

Be sure to run this code before proceeding so plot and plot2 is saved into your environment

Changing colours

To change colours of things in your ggplot you need to use the col = or the fill = arguments

```
plot+
  geom_point(shape = 21, col = "black", fill = "red")+
  theme(axis.title = element_text(colour = "red"))
```



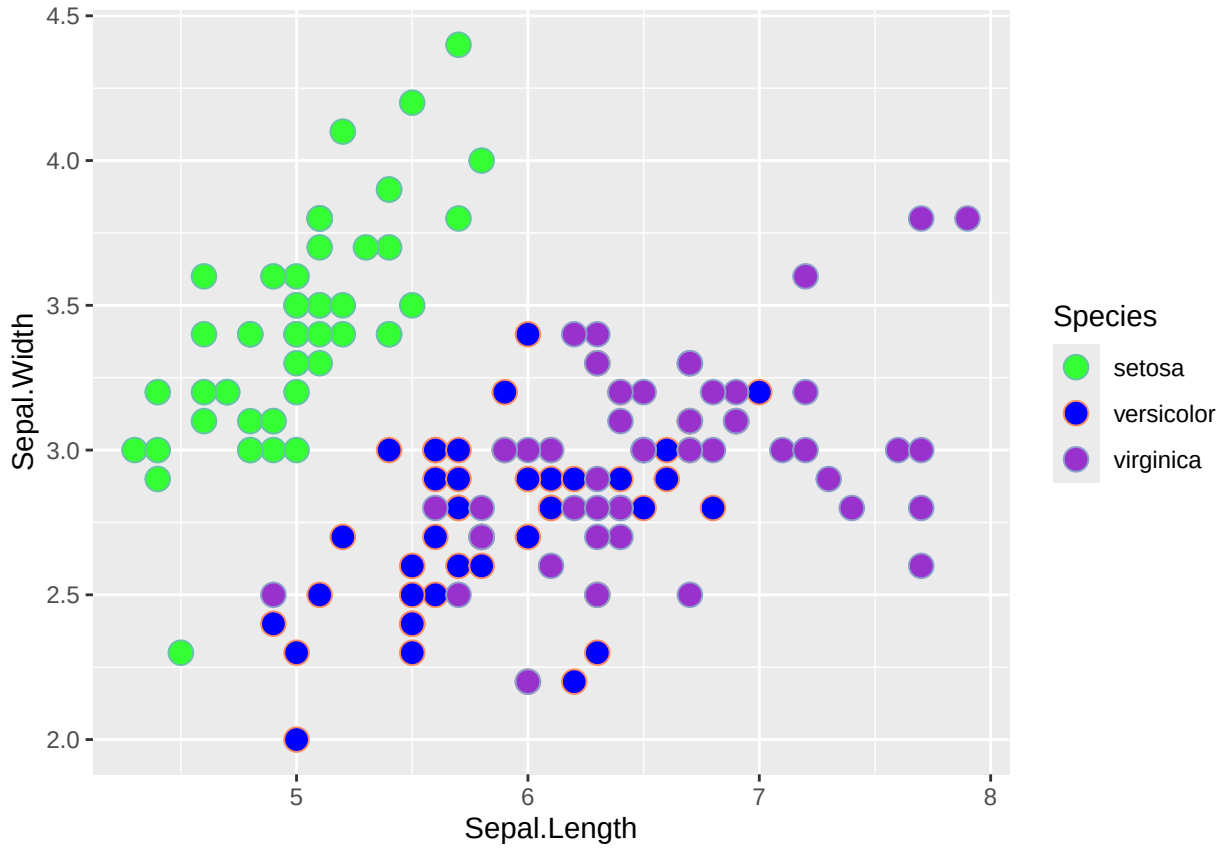
You can see here we have changed the outline and colour of the points as well as the colour of the axis titles.

You can use hex colour codes instead of the name of the colour for a more specific look to your ggplot.

The rapidtables website is useful for this.

If you want your colours to be based on another column in the data set you can do this by using the aes() function. To change these colours you can use a couple of different functions.

```
library(RColorBrewer)

plot+
  geom_point(aes(fill = Species, col = Species),
             shape = 21,
             size = 4)+
  scale_fill_manual(values = c("#33ff33", "blue", "#9932cc"))+
  scale_color_brewer(palette = "Set2")
```



The code above shows how to use scale_fill_manual and scale_col_manual as well as how to use the rcolorbrewer package to use pre-made colour palettes.

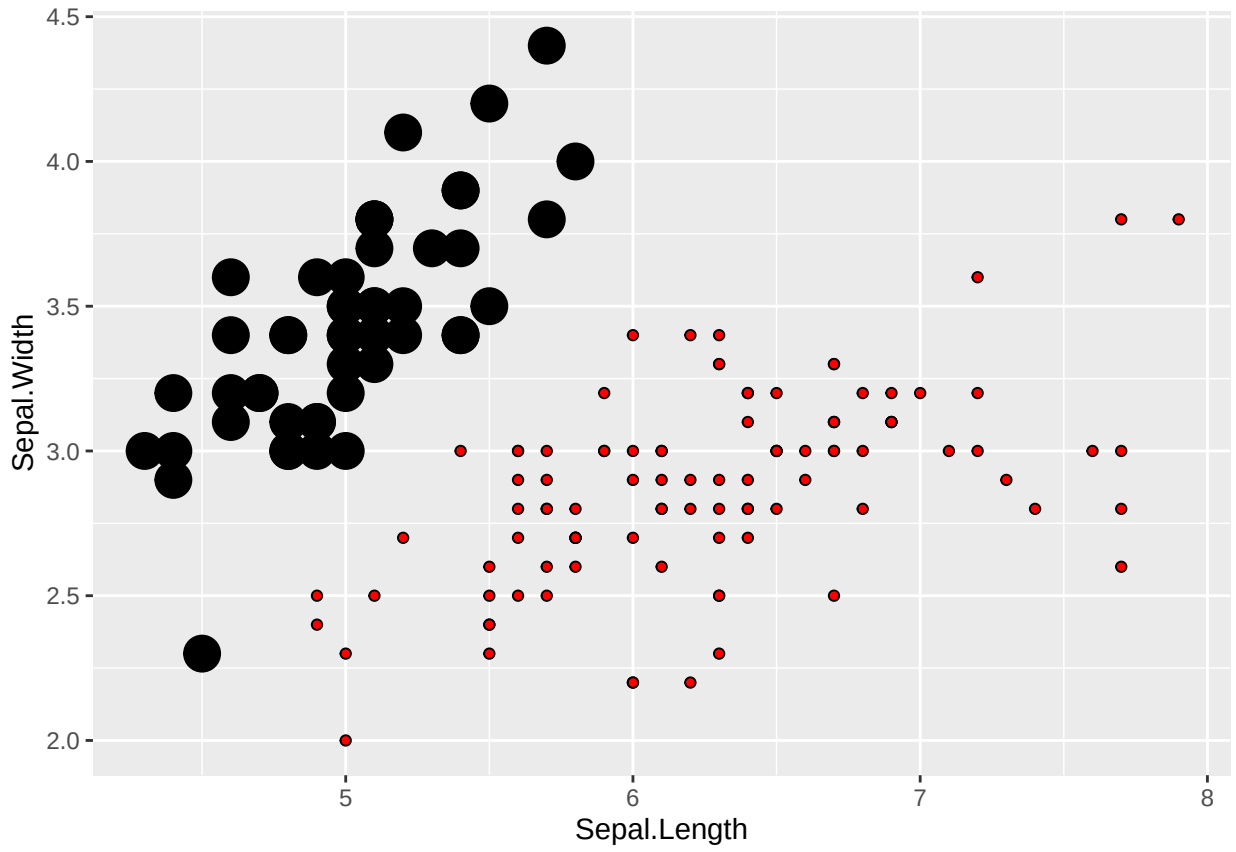
For more information on the **RColourBrewer** package click here

Make sure to install and load "RColorBrewer" before trying this.

Adding data from another dataset

Let's say you made a ggplot using one dataset and then wanted to also include some information from another dataset then this is how you would do it.

```
# load second data
data("iris3")

# make plot
plot+
  geom_point(shape = 21, col = "black", fill = "red")+
  geom_point(inherit.aes = F,
             data = as.data.frame(iris3),
             size = 6,
             aes(x = `Sepal L..Setosa`, y = `Sepal W..Setosa`))
```



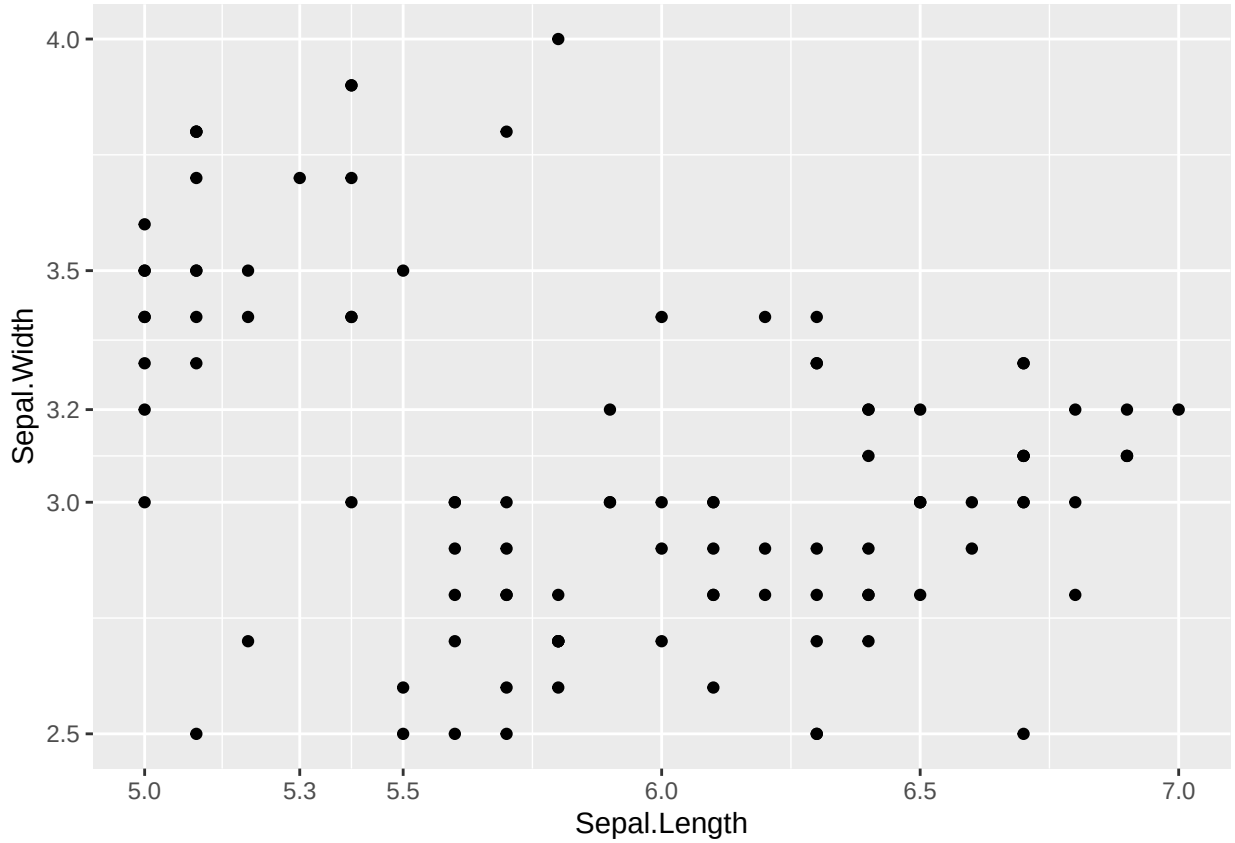
Changing axis limits and breaks

This is when you want to specify the limits of the axis and also change the breaks on the plot so that more or less numbers are shown.

```
plot+
  geom_point()+
```

```
  scale_x_continuous(breaks = c(5, 5.3, 5.5, 6, 6.5, 7),
                     limits = c(5,7))+
  scale_y_continuous(breaks = c(2.5, 3, 3.2, 3.5, 4, 4.5),
                     limits = c(2.5, 4))
```

```
## Warning: Removed 46 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



This example may seem a bit silly, however sometimes it can be useful to specify the breaks on a plot.

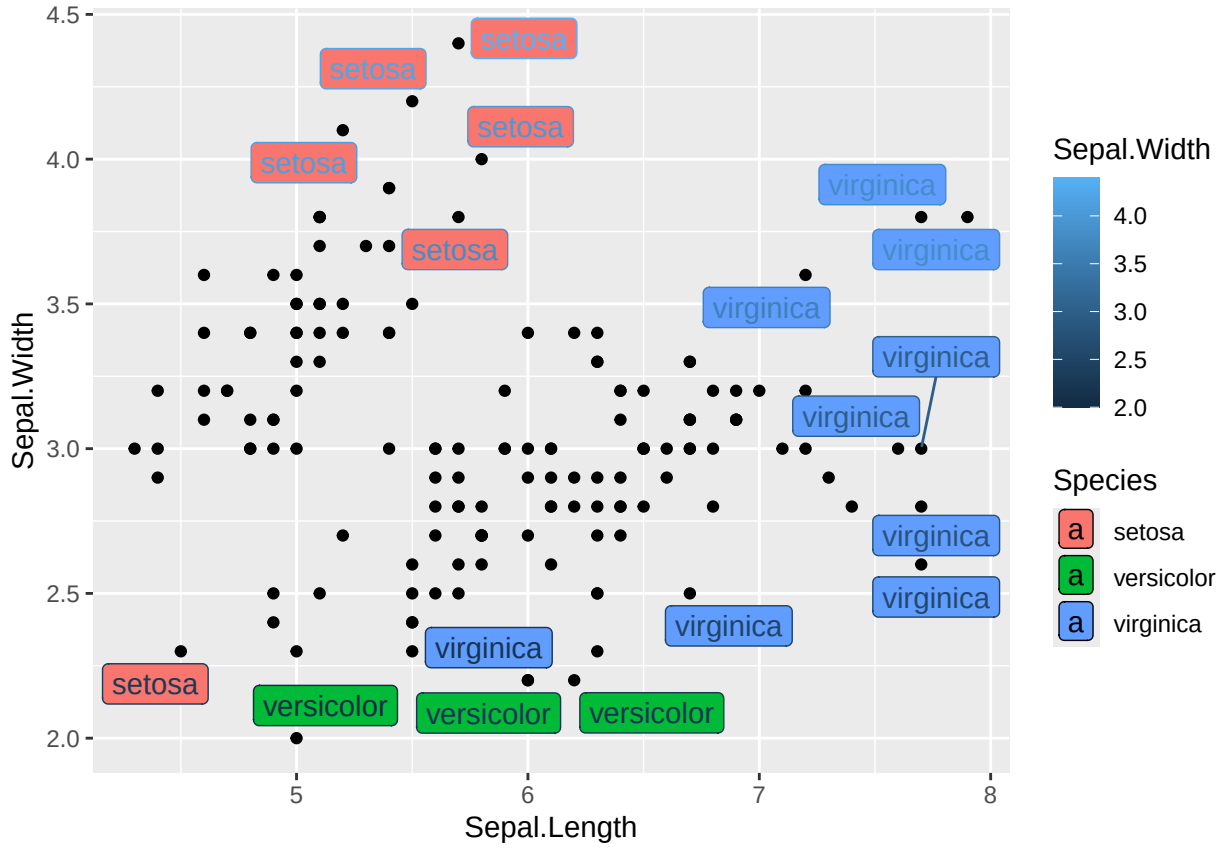
Labelling points

If you wanted to have text on a ggplot to label some information from your dataframe you can do that.

```
plot+
  geom_point()+
  geom_label(aes(label = Species,
                 fill = Species,
                 col = Sepal.Width))
```

The R package **ggrepel** has a really useful function called **geom_label_repel** which can be used to avoid overcrowding when labelling points

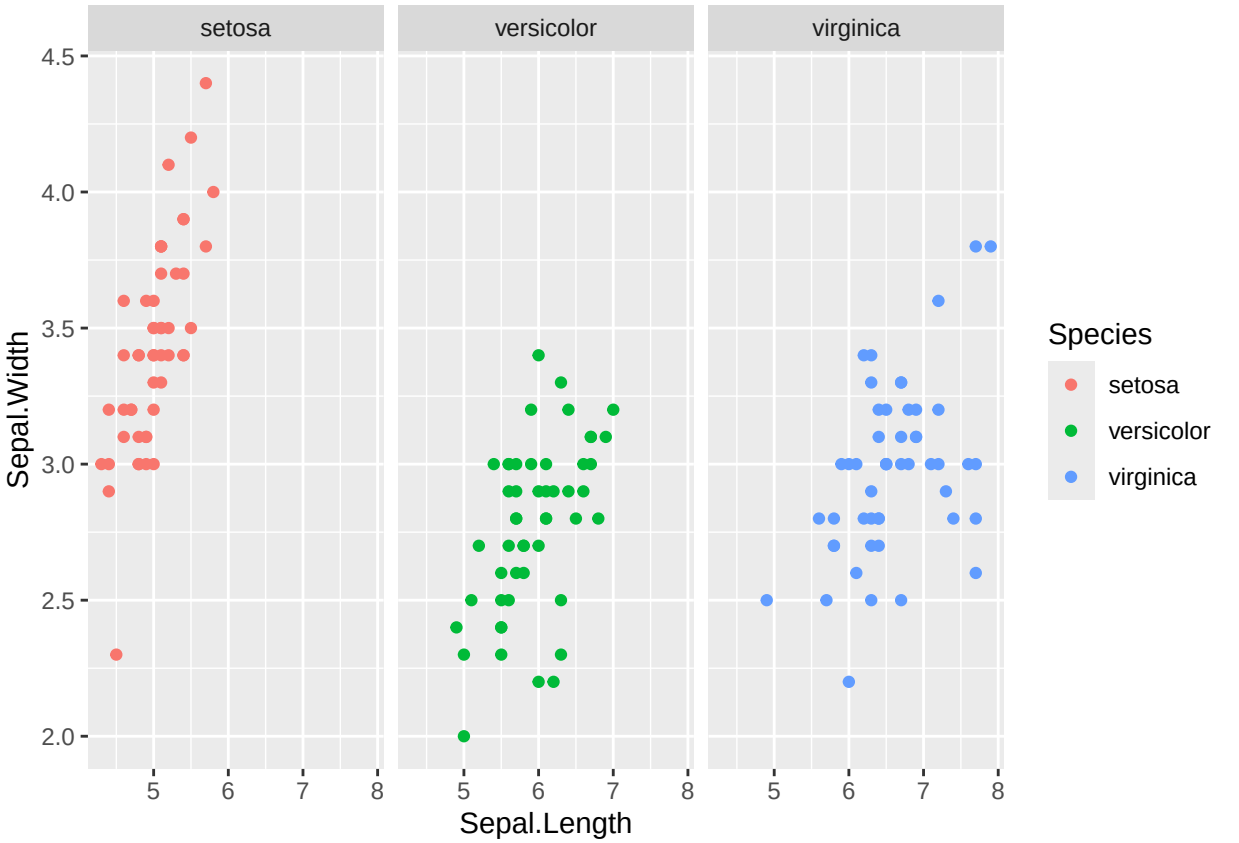
```
library(ggrepel)

plot+
  geom_point()+
  geom_label_repel(aes(label = Species,
                       fill = Species,
                       col = Sepal.Width))
```



Faceting plot

Faceting in ggplot refers to the process of creating multiple plots based on subsets of the data, all laid out together in a grid. This is particularly useful when you want to compare the same plot type (e.g., scatter plot, line plot) across different categories or levels of a variable.

```
plot+
  geom_point(aes(col = Species))+
  facet_wrap(~Species)
```



Saving ggplot images to your computer

Saving your ggplot is one of the most important steps so that you can produce high quality images.

The **ggsave()** function is perfect for this and is really useful for making nice looking plots.

Firstly you need to save the plot to your environment

```
plot_to_save <- plot+
  geom_point(aes(col = Species))
```

Then you can save the plot to your computer

```
ggsave(filename = "saved_plot.png", # the name of the file
       plot = plot_to_save, # the name of the plot
       device = "png", # type of file (jpg, png)
       height = 5, # height
       width = 5, # width
       dpi = 600, # quality 600 is good
       path = "C:/Users/username/Desktop/") # file path (need to change)
```

Dplyr cheat sheet

Dplyr is a really useful package for data cleaning and the following functions are a good start to learn more

filter() – Extract rows based on a specific condition

rename() – Rename columns in a dataframe

mutate() – Make new columns in a dataframe

summarise() – Transform data into a single row

There are many more but this is a good starting point

The final bit of information regarding dplyr is to understand how the %>% (pipe operator) works.

The pipe allows you to pass many functions at the same time. So instead of this:

```
iris_1 <- rename(iris, sepal_length = Sepal.Length)
iris_2 <- mutate(iris_1, length = sepal_length+Petal.Length)
iris_3 <- filter(iris_2, Species == "setosa")
```

You can code it like this:

```
iris_final <- iris %>%
  rename(sepal_length = Sepal.Length) %>%
  mutate(length = sepal_length+Petal.Length) %>%
  filter(Species == "setosa")
```

Both methods work however the second one is easier to write and saves less objects into the environment.

Further information

The aim of this book was to give you an introduction to plotting in ggplot2 and to hopefully improve your confidence using Rstudio.

When applying these principles to your own data make sure the data is in the same structure as the examples used in this booklet.

If you have any advice on how to improve this booklet or anything that should be included, please do not hesitate to get in touch (hlajens2@liverpool.ac.uk).